🛠️ MacVNC-Turbo: Architecture & Design

(深度架构与设计解析)

This document details the architectural decisions, technical principles, and the ultimate problem-solving journey behind **MacVNC-Turbo**. We break through the deep native protocol barriers between Windows and macOS using brutally minimalist code.

本文档详细揭秘 MacVNC-Turbo 背后的设计思想、实现原理、技术选型逻辑，以及我们在开发过程中踩过的“坑”与终极解法。我们用极简的代码击穿了 Windows 和 macOS 之间深层的原生协议壁垒。

🛑 1. Constraints & Tech Selection / 限制条件与技术选型

To understand why MacVNC-Turbo is built this way, we must first look at the strict constraints we imposed on ourselves.

要理解 MacVNC-Turbo 为什么这样设计，我们必须首先看看我们给自己设定的严苛“限制条件”。

Constraint 1: Zero footprint on macOS (macOS 端零污染)

- **Goal:** Do not install any third-party remote control software (like TeamViewer, ToDesk, or AnyDesk) on the target Mac.
- **Reality:** We must rely **exclusively** on macOS's built-in "Screen Sharing" feature, which runs an ancient, native **TCP-based VNC/RFB Server**.
- *(目标：绝对不在目标 Mac 上安装任何第三方远控软件。现实：我们只能依赖 macOS 系统自带的“屏幕共享”功能，它在底层运行的是一个基于 裸 **TCP** 协议 的 VNC/RFB 服务端。)*

Constraint 2: Client must be a Web Browser (客户端必须是纯网页)

- **Goal:** The Windows client must not require installing heavy client software. It should run instantly in Chrome or Edge.
- *(目标：Windows 控制端不需要安装任何臃肿的客户端软件，必须能在 Edge/Chrome 浏览器里即开即用。)*

The Core Conflict: Why a "Pure Web" Solution Fails (核心冲突：为什么“纯网页”行不通？)

Many wonder: *"If it's just a web client, why do we need a proxy at all? Why can't the HTML file connect to the Mac directly?"*

The answer lies in the **Browser Security Sandbox**. Modern browsers **cannot initiate raw TCP connections** (OSI Layer 4). They can only communicate via higher-level protocols like HTTP, HTTPS, WebSockets (WS), or WebRTC. However, the macOS VNC server **only understands**

raw TCP. They speak entirely different languages.

*(很多人会问："既然是网页端，为什么 HTML 不能直接连 Mac？为什么要搞个代理？"答案在于浏览器安全沙盒。现代浏览器无法发起裸 **TCP** 连接，它们只能使用 HTTP、WebSocket 等高层协议。然而，Mac 的 VNC 服务端只听得懂裸 **TCP**。两者语言根本不通。)*

Why This Solution is Optimal (为什么我们的方案是在此条件下的最优解)

To bridge a WebSocket (Browser) and TCP (Mac), a protocol proxy is mathematically required.

Traditional solutions use tools like websockify (written in Python or Node.js). But requiring a Windows user to install a 200MB Python environment just to run a proxy violates our "minimalism" rule.

Our Optimal Solution: We wrote a micro-proxy in pure C++ using native Windows APIs (Winsock2 and Wincrypt). It compiles to a tiny standalone executable (~100KB) with **zero dependencies**. It perfectly bridges the gap without any bloated runtimes.

(为了连接 WebSocket 和 TCP，必须有一个"翻译官"代理。传统方案通常用 Python 或 Node.js 跑一个 websockify，但这需要用户安装几百兆的运行环境，严重违背了我们的"极简"初衷。我们的最优解：直接调用 Windows 底层原生 API，用纯 C++ 手搓了一个微型代理。编译后仅几十 KB，无任何环境依赖，完美填补了协议鸿沟。)

🏗️ 2. Technical Principles & Architecture / 实现技术原理

The system operates strictly across three layers without central servers (P2P LAN).

系统在没有中心服务器的情况下跨越三层运行（局域网直连）。以下是系统的数据流转架构：

💻 **Windows Host (宿主机)**	↔️ **通信链路 (Protocol)**	🍏 **macOS Target (目标机)**
① 浏览器 **(Web Browser)** • HTML5 Canvas + noVNC • 强制 GPU 硬件渲染	↕️ **WebSocket (WS)** *(127.0.0.1:6080)* ↕️	
② 核心代理 **(MacVNC-Turbo)** • C++ 微型可执行文件	↔️ 原生裸 **TCP** *(Mac_IP:5900)*	③ **Mac** 系统服务 **(Native VNC)** • 原生屏幕共享 (RFB Server)

• 0 依赖，内存级封包转换	↔️	• Apple Auth 协议验证

1. **Frontend Layer (前端层):** A static HTML file utilizing the noVNC core. It translates user mouse/keyboard inputs into RFB (Remote Framebuffer) protocol payloads, wrapped inside WebSocket frames, and renders incoming pixel data onto a high-performance HTML5 <canvas>.
   (纯静态 HTML。它将鼠标/键盘动作翻译为 RFB 协议数据，包裹在 WebSocket 帧中发送；并将收到的像素数据渲染在高性能 Canvas 上。)
2. **Bridge Layer (桥接层 - The C++ Proxy):** The proxy listens for incoming browser connections. It intercepts the HTTP Upgrade: websocket handshake, uses Wincrypt to calculate the Sec-WebSocket-Accept hash, and establishes the WebSocket tunnel. It then opens a raw TCP socket to the Mac's IP (Port 5900). It acts as a bidirectional pipe: unmasking WS frames and forwarding raw bytes to TCP, and vice versa.
   (C++ 代理监听浏览器请求，用底层加密库计算并完成 WebSocket 握手。随后它向 Mac 的 5900 端口发起 TCP 连接。它充当一个双向水管：解包浏览器的 WS 数据发给 Mac (TCP)，并将 Mac 的画面数据打包成 WS 帧发给浏览器。)
3. **Target Layer (目标层):** The native macOS screen sharing service blindly accepts the incoming TCP connection, unaware that the client is actually a Windows web browser.
   (Mac 原生屏幕共享服务直接接收 TCP 连接，完全不知道控制它的其实是个 Windows 浏览器。)

✨ 3. Core Architecture Highlights / 核心架构亮点

3.1 Brutal Minimalism (极致的轻量与零依赖)

Zero Python, Zero Node.js. Compiled via MSVC, the proxy utilizes ws2_32.lib and crypt32.lib. It is a truly standalone binary.

(拒绝臃肿环境。C++ 代理仅调用 Windows 原生网络和加密 API，是一个真正的独立可执行文件。)

3.2 Ultra-Performance Engine (榨干底层的传输性能)

To handle the massive uncompressed pixel data from 4K/5K Retina displays:

(为了应对 Mac 视网膜屏幕庞大的像素数据，我们在底层做了极限压榨：)

- **Disabled Nagle's Algorithm (TCP_NODELAY):** Forces instant transmission of tiny packets (like mouse movements), achieving nanosecond responsiveness. *(关闭 Nagle 算法，拒绝数据堆积，实现纳秒级鼠标响应。)*
- **Zero-Copy In-Place Framing (内存级就地封包):** Instead of copying massive video payloads into new WebSocket frames, we pre-allocate 8MB buffers and write the WebSocket header *directly* into the memory offset ahead of the TCP payload, sending it in one single syscall. *(摒弃动态内存拷贝，直接在 8MB 内存池中进行偏移组装，单次吞吐量高*

达 *4MB*。*)*

3.3 Overdrive Mode: Hardware Rendering (硬件级渲染接管)

Single-threaded JavaScript struggles to paint millions of pixels per second. By injecting transform: translate3d(0, 0, 0) and will-change: transform into the CSS, we force the browser to hand the Canvas over to the dedicated Windows GPU (Discrete Graphics) as a 3D texture for rasterization.

(为了拯救 JS 单线程渲染 4K 画面的吃力表现，前端利用 CSS 3D 属性强制浏览器跳过 2D 引擎，直接将 Canvas 交给 Windows 独立显卡进行光栅化渲染。)

🚧 4. Challenges & Ultimate Solutions / 遇到的技术挑战与终极解法

🩸 Challenge 1: The Apple Auth Trap (Apple 专属加密的“二次验证死锁”)

- **Issue:** Standard VNC only asks for a password. macOS enforces Apple Remote Desktop (ARD) encryption (Type 30/35), which requires **both a username and a password**. Traditional Web VNC clients freeze at "Connecting..." because they don't know how to prompt for a username.
- **Solution:** We intercept the internal credentialsrequired event. When the Mac demands username, we dynamically trigger a prompt, assemble the proprietary Apple Auth payload, and breach the security wall.
- *(问题：Mac 强制使用苹果专有加密，索要用户名+密码，导致传统客户端卡死。解法：前端动态监听凭据事件，拦截并弹出二次输入框，打通 Apple 验证。)*

🩸 Challenge 2: 64KB Frame Overflow (16位 WebSocket 长度溢出导致的断流)

- **Issue:** Uncompressed Retina frames easily exceed hundreds of kilobytes. The standard 16-bit WebSocket length indicator maxes out at 65,535 bytes. Exceeding this causes the payload size to overflow to 0, causing the browser to instantly drop the connection due to protocol corruption.
- **Solution:** We manually implemented the 64-bit Extended Payload standard (RFC 6455) in the C++ proxy. Using 8-byte bitwise shifts, it flawlessly supports transmitting up to 16 Exabytes per frame, curing the stream dropouts.
- *(问题：单帧高清画面经常突破 64KB，导致 16位 WebSocket 长度标记溢出清零，连接瞬间崩塌。解法：在 C++ 中纯手搓 64 位超大帧头封装协议，完美支持 GB 级超大数据包传输。)*

🩸 Challenge 3: The Double-Typing Race (跨平台组合键的 DOM 抢跑冲突)

- **Issue:** Windows users press Ctrl+C, but macOS expects Cmd+C. Trying to intercept this at the DOM level caused an "Event Race": our code sent a C, and the underlying noVNC

library also sent a C, resulting in double-typing bugs.

- **Solution (The Flawless Monkey-Patch):** We abandoned unstable DOM interception. Instead, we applied an "Arguments-Preserving Monkey Patch" deep inside noVNC's rfb.sendKey function. Just microseconds before the packet leaves for the Mac, we intercept it in memory, swapping 0xFFE3 (Win Ctrl) for 0xFFEB (Mac Cmd), leaving regular typing completely untouched. Perfect muscle memory, zero bugs.
- *(问题：在网页层拦截 Ctrl 键会导致原生事件库发生“抢跑”，打出两个字母。解法：放弃 DOM 拦截，直接深入底层发包函数打“猴子补丁”，在发往 Mac 的最后一微秒瞬间将 Ctrl 键的 Hex 码替换为 Cmd，彻底消灭双击 Bug。)*

🩸 Challenge 4: Downsampling Disaster (Retina 缩放的性能灾难)

- **Issue:** Squeezing a 4K Mac display into a 1080P Windows browser window forces the browser to perform incredibly expensive real-time downsampling and anti-aliasing calculations, which is the primary cause of VNC lag.
- **Solution:** Introduced Pixel-Perfect (1:1 scale) and Overdrive modes. We disabled browser auto-scaling and injected image-rendering: pixelated to kill bilinear interpolation. By advising users to set the Mac to 1080P HiDPI, payload size drops by 75%, and rendering lag drops to zero.
- *(问题：将 4K 画面强行塞入 1080P 窗口会导致极其昂贵的实时缩放抗锯齿计算，严重拖垮帧率。解法：引入 Pixel-Perfect 原画模式，关闭浏览器缩放与双线性插值。当发包量锐减且浏览器免于缩放计算时，体验直接起飞。)*